

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Короткевич Анастасия

Группа

J3212

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

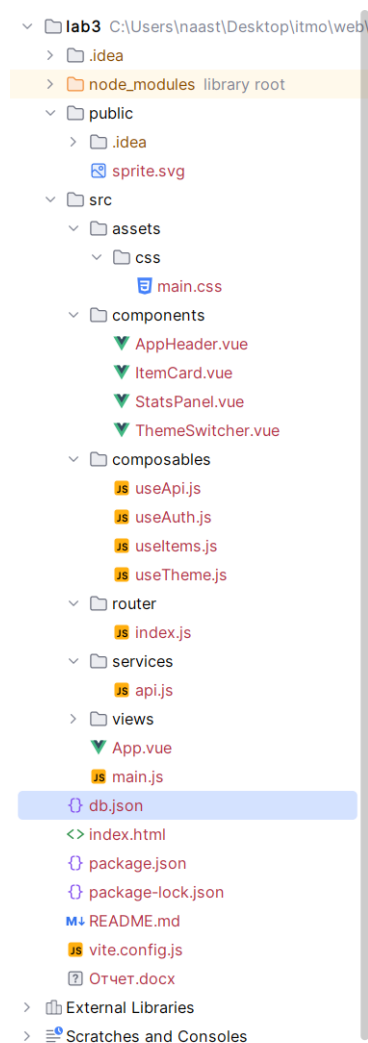
Цель: миграция ранее разработанного приложения ModelHub на фреймворк Vue.JS 3.

В ходе работы необходимо выполнить следующие задачи:

- Настроить Vue Router для навигации между страницами без перезагрузки
- Реализовать работу с внешним API посредством axios
- Разделить проект на компоненты и страницы (views)

Ход работы

1. Архитектура проекта



2. Vue Router

В index.js создан router с history-режимом, определены маршруты и catch-all, который перенаправляет на главную страницу

```
1 import { createRouter, createWebHistory } from 'vue-router' ✓
2 import HomeView from '../views/HomeView.vue'
3 import SearchView from '../views/SearchView.vue'
4 import ProfileView from '../views/ProfileView.vue'
5 import LoginView from '../views/LoginView.vue'
6 import RegisterView from '../views/RegisterView.vue'
7 import ModelView from '../views/ModelView.vue'
8
9 const routes : [{path: string, name: string, ...} = [
10   { path: '/', name: 'home', component: HomeView },
11   { path: '/search', name: 'search', component: SearchView },
12   { path: '/profile', name: 'profile', component: ProfileView },
13   { path: '/login', name: 'login', component: LoginView },
14   { path: '/register', name: 'register', component: RegisterView },
15   { path: '/item/:type/:id', name: 'item', component: ModelView, props: true },
16   { path: '/*', name: 'catch-all', redirect: '/' }
17 ]
18
19 export default createRouter( options: { history: createWebHistory(), routes } ) Show usage
20
```

Здесь маршрут использует динамические сегменты type и id, которые передаются в компоненты как props (props: true). Благодаря чему modelview получает параметры url без обращения к useroute

3. Axios

В api.js создан axios-экземпляр с базовым url mock-сервера. Добавлен request-интерцептор, который автоматически подставляет bearer-токен из localStorage в заголовок авторизации каждого запроса:

```
import axios from 'axios'

const api : AxiosInstance = axios.create({ baseURL: 'http://127.0.0.1:3000' })

api.interceptors.request.use( onFulfilled: (config) => {
  const token :string = localStorage.getItem( key: 'accessToken' )
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

export default api Show usages
```

Все Api-запросы:

Метод	Эндпоинт	Назначение
GET	/models	Список всех моделей (HomeView, SearchView)
GET	/datasets	Список всех датасетов
GET	/models/:id	Карточка конкретной модели (ModelView)
GET	/datasets/:id	Карточка конкретного датасета
GET	/users?email=...	Поиск пользователя при логине (useAuth)
GET	/users/:id	Профиль пользователя (ProfileView, ModelView)
POST	/users	Регистрация нового пользователя
PATCH	/users/:id	Обновление профиля (ProfileView)
PATCH	/models/:id	Инкремент stars/downloads/forks
PATCH	/datasets/:id	Инкремент stars/downloads/forks
GET	/discussions	Комментарии к объекту (ModelView)
POST	/discussions	Добавление комментария

4. useApi.js

Общая обертка для любого api-запроса. Хранит состояния data, loading, error. Можно использовать при необходимости дополнительного контроля над состоянием загрузки

```

1 import { ref } from 'vue'
2
3 export function useApi(request) : {...} { no usages
4   const data :[*] extends [Ref] ? IfAny<null...> = ref( value: null)
5   const loading :Ref<UnwrapRef<...>, ...> = ref( value: false)
6   const error :Ref<UnwrapRef<...>, ...> = ref( value: '')
7
8   const execute = async (...args) :Promise<...> => { Show usages
9     loading.value = true
10    error.value = ''
11    try {
12      const response = await request(...args)
13      data.value = response.data
14      return response.data
15    } catch (err) {
16      error.value = err.response?.data || err.message || 'Ошибка загрузки данных'
17      throw err
18    } finally {
19      loading.value = false
20    }
21  }
22
23  return { data, loading, error, execute }
24 }
25

```

5. useAuth.js

управляет состоянием авторизации: хранит userId, username, userAvatar как реактивные реф-переменные, которые синхронизируются с localStorage. Использует методы login, register, logout, updateUserMeta. Используется в AppHeader, LoginView, RegisterView и ProfileView.

```

export function useAuth() : {...} { Show usages
  const isAuth :ComputedRef<boolean> = computed( getter: () :any | boolean => Boolean(localStorage.getItem( key: 'accessToken')))

  const login = async (email, password) :Promise<any> => { Show usages
    const users :AxiosResponse<any> = await api.get( url: '/users', config: { params: { email } })
    const user = users.data[0]
    if (!user) throw new Error('Пользователь не найден')
    localStorage.setItem('accessToken', 'demo-token')
    localStorage.setItem('userId', user.id)
    localStorage.setItem('userName', user.name)
    localStorage.setItem('userAvatar', user.avatar || '')
    userId.value = user.id
    userName.value = user.name
    userAvatar.value = user.avatar || ''
    return user
  }
}

```

6. useItems.js

Инкапсулирует загрузку всех моделей и датасетов через параллельные запросы (Promise.all), а также методы getItem и updateStat. Используется в HomeView, SearchView, ModelView и ProfileView.

```
4 export function useItems() : {...} { Show usages
5   const models : Ref<UnwrapRef<...>, ...> = ref( value: [] )
6   const datasets : Ref<UnwrapRef<...>, ...> = ref( value: [] )
7
8   const loadItems = async () : Promise<void> => { Show usages
9     const [modelsResponse : AxiosResponse<any>, datasetsResponse : AxiosResponse<any>] = await Promise.all( values: [
10       api.get( url: '/models' ),
11       api.get( url: '/datasets' )
12     ])
13     models.value = modelsResponse.data
14     datasets.value = datasetsResponse.data
15   }
16
17   const allItems : ComputedRef<...> = computed( getter: () => [...models.value, ...datasets.value] )
18
19   const getItem = async (type, id) : Promise<any> => { Show usages
20     const resource : string = type === 'dataset' ? 'datasets' : 'models'
21     const response : AxiosResponse<any> = await api.get( url: `/${resource}/${id}` )
22     return response.data
23   }
24
25   const updateStat = async (item, field) : Promise<any> => { Show usages
26     const resource : string = item.type === 'dataset' ? 'datasets' : 'models'
27     const nextValue : number = Number( value: item[field] || 0 ) + 1
28     const response : AxiosResponse<any> = await api.patch( url: `/${resource}/${item.id}`, data: { [field]: nextValue } )
29     return response.data
30   }
31
32   return { models, datasets, allItems, loadItems, getItem, updateStat }
33 }
34
```

7. useTheme.js

сохраняет текущую тему (светлую, темную, фиолетовую) в реактивной реф-переменной, синхронизированной с localStorage. Используется в App.vue и ThemeSwitcher.vue.

```
1 import { ref } from 'vue'
2
3 const theme : Ref<UnwrapRef<...>, ...> = ref( value: localStorage.getItem( key: 'theme' ) || 'light' )
4
5 export function useTheme() : {setTheme: function(any): void, theme: [any] extends [Ref] ? IfAny<an...> } { Show usages
6   const setTheme = (value) : void => { Show usages
7     theme.value = value
8     localStorage.setItem( 'theme', value )
9   }
10   return { theme, setTheme }
11 }
12
```

8. Компоненты

- App.vue – скелет приложения. Подключает шапку, точку монтирования роутера и переключатель темы. Получает текущую тему из useTheme() и применяет её через :data-theme.
- AppHeader.vue – отображает шапку сайта, где расположены логотип, поисковик и навигация. При отправке формы перенаправляет на /search?q= Если пользователь авторизован, показывает аватар, иначе кнопку «Войти».
- ItemCard.vue – карточка модели или датасета. Принимает один prop item (required Object). Содержит заголовок, метаданные, описание и статистику. Оборачивается в RouterLink.
- StatsPanel.vue - вычисляет и отображает суммарные загрузки, звёзды и форки по массиву items.
- ThemeSwitcher.vue – виджет с тремя темами.

9. Страницы

View	Маршрут
HomeView	/
SearchView	/search
ModelView	/item/:type/:id
ProfileView	/profile
LoginView	/login
RegisterView	/register

Вывод

Вывод по работе было перенесено приложение на vue 3.